

Q1. Explain the difference between frontend, backend, and full-stack development with suitable real-world examples.

Web development is broadly divided into three domains based on which part of an application a developer works on: Frontend, Backend, and Full-Stack development. Each domain uses a distinct set of technologies and is responsible for a distinct part of the overall system.

1. Frontend Development

Frontend development (also called client-side development) deals with everything the user directly sees and interacts with in a browser — layout, design, buttons, forms, navigation menus, and responsiveness across devices.

Core Technologies: HTML (structure), CSS (styling), JavaScript (interactivity), and modern frameworks/libraries such as React.js, Angular, and Vue.js.

Example: On Amazon's website, the product image gallery, the "Add to Cart" button, the search bar, and the responsive layout that adapts to mobile and desktop screens are all built and maintained by frontend developers.

2. Backend Development

Backend development (server-side development) deals with the logic, database interactions, authentication, and application processing that happen behind the scenes, away from direct user view.

Core Technologies: Server-side languages and frameworks such as Node.js (Express), Python (Django/Flask), Java (Spring Boot), PHP (Laravel), along with databases such as MySQL and MongoDB.

Example: When a user clicks "Add to Cart" on Amazon, the backend verifies stock availability, calculates the price with applicable discounts, stores the order in the database, and later processes secure payment through a payment gateway.

3. Full-Stack Development

A full-stack developer is proficient in both frontend and backend development and can build a complete, end-to-end web application independently — from the user interface to the server logic and database.

Core Technologies: A common combination is the MERN stack — MongoDB, Express.js, React.js, and Node.js — which allows a single developer to handle the entire application stack.

Example: A solo developer or a small startup team building a complete food-delivery application like Zomato — designing the app's UI, writing the order-processing logic, and managing the restaurant/menu database — is performing full-stack development.

Summary Table

Aspect	Frontend	Backend	Full-Stack
Focus	User Interface & Experience	Server logic, database, APIs	Both Frontend + Backend
Languages	HTML, CSS, JavaScript	Node.js, Python, Java, PHP	Combination of both
Visibility	Visible to the user	Not visible to the user	Covers both layers
Example Role	UI/UX implementation	API & database management	End-to-end app development

Q2. Create a simple diagram showing how the client-server model works in web architecture.

The client-server model is the fundamental architecture of the web, where a client (such as a web browser) requests resources or services, and a server processes the request and returns the appropriate response. The diagram below illustrates this two-way communication.

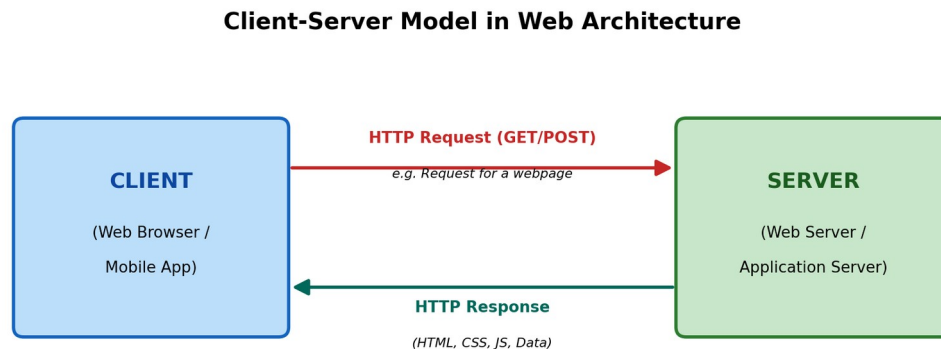


Fig 1: The client sends a request over the Internet; the server processes it and sends back a response.

As shown in the figure, the Client (browser or mobile app) sends an HTTP Request to the Server asking for a resource (for example, a web page). The Server processes this request — which may involve fetching data from a database or performing computation — and sends back an HTTP Response containing HTML, CSS, JavaScript, or other data. The browser then renders this response for the user.

Q3. Describe how a browser requests and displays a web page from a web server.

When a user wants to view a web page, the browser goes through a well-defined sequence of steps to fetch and render that page:

- Step 1 — URL Entry: The user types a URL (e.g., `www.example.com`) into the browser's address bar and presses Enter.
- Step 2 — DNS Lookup: The browser contacts a DNS (Domain Name System) server to resolve the human-readable domain name into its corresponding numerical IP address.
- Step 3 — Establishing a Connection: The browser opens a TCP connection with the web server at that IP address, typically via a three-way handshake (SYN, SYN-ACK, ACK). For secure sites, a TLS/SSL handshake is also performed (HTTPS).
- Step 4 — Sending the HTTP Request: The browser sends an HTTP GET request to the server, asking for the specific resource (e.g., the `index.html` file).
- Step 5 — Server Processing: The web server receives the request, locates or generates the requested resource — this may involve querying a database or running backend application code — and prepares an HTTP response.
- Step 6 — Sending the HTTP Response: The server sends back the response, which includes a status code (e.g., 200 OK), response headers, and the actual content (HTML, CSS, JavaScript, images, etc.).
- Step 7 — Rendering the Page: The browser's rendering engine parses the HTML to build the DOM (Document Object Model), parses CSS to build the CSSOM, combines them into a render tree, applies layout calculations, and finally paints the pixels on screen. JavaScript is executed to add interactivity.

This entire process — from typing a URL to seeing the fully rendered page — usually takes only a fraction of a second on a fast connection.

Q4. Identify and list the tools required to set up a web development environment. Explain the purpose of each.

Tool	Purpose
Code Editor (e.g., VS Code, Sublime Text)	Used to write, edit, and organize source code (HTML, CSS, JS, backend code) with features like syntax highlighting, IntelliSense, and extensions.
Web Browser (e.g., Chrome, Firefox)	Used to run and visually test the web application; browser Developer Tools help inspect elements, debug JavaScript, and monitor network requests.
Version Control System (Git) + GitHub/GitLab	Tracks changes to source code over time, enables collaboration among multiple developers, and allows reverting to previous versions if needed.
Node.js and npm	Node.js provides a JavaScript runtime to run JS outside the browser; npm (Node Package Manager) is used to install and manage third-party libraries and frameworks.
Web Server / Local Server (Apache, Nginx, XAMPP, Live Server)	Hosts the website locally or in production so that it can be accessed over HTTP by a browser.
Database Software (MySQL, MongoDB, PostgreSQL)	Stores, organizes, and manages the application's persistent data such as user accounts, products, or orders.
Browser Developer Tools (DevTools)	Built into browsers; used to inspect and debug HTML/CSS/JavaScript, view console errors, and analyze network traffic.
Terminal / Command Line Interface	Used to run commands such as installing packages (npm install), running servers, and executing Git commands.
Package/Build Tools (Webpack, Vite, Parcel)	Bundle, minify, and optimize project files (JS, CSS, images) for faster loading and production deployment.
API Testing Tool (Postman)	Used to test backend APIs by sending sample HTTP requests (GET, POST, PUT, DELETE) and checking the responses.

Q5. Explain what a web server is and give examples of commonly used servers.

A web server is a software application (often running on dedicated hardware) that stores, processes, and delivers web pages and other resources to clients over the Hypertext Transfer Protocol (HTTP) or HTTPS. When a browser requests a specific URL, the web server locates the corresponding file or dynamically generates the content and sends it back as an HTTP response. In addition to serving static files (HTML, CSS, images), many web servers work alongside application servers to execute backend logic and interact with databases for dynamic content.

Commonly Used Web Servers

Web Server	Description
Apache HTTP Server	One of the oldest and most widely used open-source web servers, known for stability, flexibility, and a large module ecosystem.
Nginx	A high-performance, open-source web server known for handling large numbers of concurrent connections efficiently; also used as a reverse proxy and load balancer.

Web Server	Description
Microsoft IIS (Internet Information Services)	A web server built for and tightly integrated with Windows Server environments, commonly used for hosting ASP.NET applications.
Node.js (with Express.js)	A lightweight, JavaScript-based server environment that allows developers to build custom web servers and APIs using the same language as the frontend.
LiteSpeed Web Server	A high-performance, commercial web server known for its speed and popularity in WordPress and shared hosting environments.

Q6. Define the roles of a frontend developer, backend developer, and database administrator in a project.

Frontend Developer

A frontend developer is responsible for translating design mock-ups (created by UI/UX designers) into functional, interactive web pages. Their responsibilities include implementing responsive layouts, ensuring cross-browser compatibility, optimizing page-load performance, handling client-side form validation, and connecting the interface to backend APIs to display dynamic data to the user.

Backend Developer

A backend developer builds and maintains the server-side logic that powers the application. This includes designing and implementing RESTful or GraphQL APIs, writing business logic, handling user authentication and authorization, managing sessions and security, integrating third-party services, and ensuring the application can scale to handle multiple simultaneous users.

Database Administrator (DBA)

A database administrator is responsible for the design, implementation, performance, and security of the project's database systems. Key responsibilities include designing efficient database schemas, writing and optimizing queries, managing indexes, configuring regular backups and disaster recovery, controlling user access permissions, and monitoring database performance to prevent bottlenecks.

In a typical project, these three roles work closely together: the frontend developer builds what the user sees, the backend developer provides the logic and APIs that connect the frontend to the data, and the DBA ensures that the underlying data is stored safely, efficiently, and remains available.

Q7. Install VS Code and configure it for HTML, CSS, and JavaScript development. Take a screenshot of the setup.

Visual Studio Code (VS Code) is a free, lightweight, and highly extensible source-code editor developed by Microsoft, widely used for web development. Follow the steps below to install and configure it:

- Step 1: Visit the official website at <https://code.visualstudio.com/> and download the installer for your operating system (Windows / macOS / Linux).
- Step 2: Run the installer and follow the setup wizard (Next → Accept Agreement → Install → Finish). It is recommended to check "Add to PATH" during installation on Windows.
- Step 3: Open VS Code and go to the Extensions panel (Ctrl+Shift+X or Cmd+Shift+X) and install the following recommended extensions:
 - Live Server — runs an HTML file on a local development server with automatic browser refresh on save.
 - Prettier – Code Formatter — automatically formats HTML, CSS, and JavaScript code for consistent style.
 - HTML CSS Support — provides autocomplete and IntelliSense for CSS class/id names inside HTML files.

- JavaScript (ES6) code snippets — provides handy code snippets for modern JavaScript syntax.
- Step 4: Create a new project folder, and inside it create three files: index.html, style.css, and script.js, and link the CSS and JS files inside the HTML file using <link> and <script> tags respectively.
- Step 5: Right-click on index.html in the VS Code file explorer and select "Open with Live Server" to preview the page in the browser with live reload enabled.

Note: Since this step involves a physical installation on your own computer, please install VS Code as described above and insert a screenshot of your configured editor (showing the installed extensions and your project files open) at this point in the document before final submission.

Q8. Explain the difference between static and dynamic websites. Provide an example of each.

Static Websites

A static website consists of fixed content written directly in HTML and CSS files. The content displayed is the same for every visitor and does not change unless a developer manually edits the source code and re-uploads the files. Static websites are simple, fast to load, and do not require server-side processing or a database.

Example: A simple personal portfolio website showing a fixed resume, project list, and contact information is a static website — the content remains the same for every visitor.

Dynamic Websites

A dynamic website generates its content on the fly, typically by retrieving data from a database and processing it through server-side (backend) code before sending it to the browser. The content can change based on user interaction, login status, time, or other conditions, and is usually managed through an admin panel rather than by editing raw code.

Example: Facebook is a dynamic website — each user's news feed, profile, and notifications are generated uniquely based on their account data, retrieved live from a database.

Feature	Static Website	Dynamic Website
Content	Fixed, hard-coded in HTML	Generated dynamically from a database
Technology Used	HTML, CSS (and minimal JS)	HTML, CSS, JS + backend language + database
Update Process	Requires editing source code manually	Can be updated via an admin panel/CMS
Personalization	Not possible	Possible (per-user content)
Example	Personal portfolio site	Facebook, Amazon

Q9. Research and list five web browsers. Explain how rendering engines differ between them.

A rendering engine (also called a browser engine or layout engine) is the software component within a browser that interprets HTML and CSS and converts them into the visual page displayed to the user. Different browsers use different rendering engines, which can occasionally cause the same web page to appear slightly differently across browsers.

Browser	Rendering Engine	Developed By
Google Chrome	Blink	Google
Mozilla Firefox	Gecko	Mozilla Foundation
Apple Safari	WebKit	Apple

Browser	Rendering Engine	Developed By
Microsoft Edge	Blink (Chromium-based since 2020)	Microsoft
Opera	Blink	Opera Software

Historically, Blink was forked from WebKit in 2013 by Google, which is why Chrome, Edge, and Opera (all Blink-based) tend to render pages very similarly to one another. Gecko, used by Firefox, is an independently developed engine, so it can interpret certain advanced or newer CSS/JavaScript features slightly differently, occasionally leading to minor visual or behavioral differences on the same web page. Because of these differences, developers perform cross-browser testing to ensure a website looks and functions consistently across Chrome, Firefox, Safari, Edge, and other browsers.

Q10. Draw a labeled diagram showing the basic web architecture flow — client, server, database, and APIs.

The following diagram illustrates the basic flow of a typical modern web application, showing how the Client, Server, Database, and external APIs interact with one another:

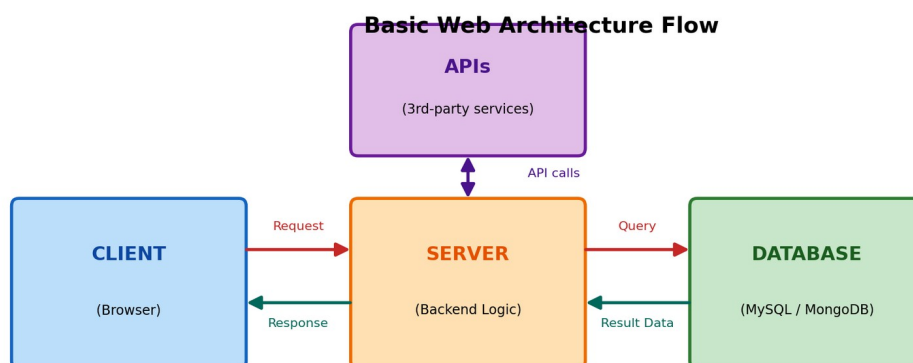


Fig 2: Client sends requests to the Server; Server fetches/stores data in the Database and may call external APIs, then sends the response back to the Client.

As shown in the diagram, the Client sends a request to the Server. The Server contains the application's business logic; it queries the Database to fetch or store data (e.g., user records, product listings) and receives the result back. In many modern applications, the Server also communicates with external APIs (such as a payment gateway or a weather service) to fetch additional data or perform third-party operations. Once all required data has been gathered, the Server sends the final response back to the Client, which then renders it for the user.